

Automata Theory

Unit-1

Lecture-1

Introduction to Automata Theory

B.Tech (CSE/IT)

AUTOMATA THEORY

Introduction and Motivation

Course: Automata Theory for Compiler Design

Programme: B.Tech (CSE/IT)

Unit: I – Introduction to Automata Theory

Lecture: 1

Topics Covered

- Why Automata Theory?
- Motivation
- Concept of Computation
- What is an Automaton?
- Historical Background
- Applications
- Compiler Design Connection

Key Idea

Automata Theory provides the mathematical foundation for understanding computation and compiler design.

Can a Computer Understand a Programming Language Directly?

Example

```
int sum = a + b;
```

Question

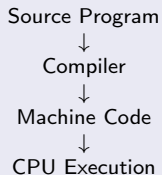
Can a computer execute this statement directly?

Answer: No

Reason

A computer understands only Machine Language (Binary: 0 and 1).

Translation Process



Key Point

Compiler converts human-readable programs into machine-understandable instructions.

What is Computation?

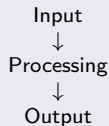
Definition

Computation is the process of transforming input into desired output by executing a finite sequence of well-defined instructions.

Components

- Input
- Processing
- Decision Making
- Output

General Computational Model



Examples

- Calculator Operation
- Google Search
- Compiler Processing

What is an Automaton?

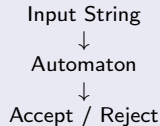
Definition

An automaton is an abstract mathematical model of computation that reads input symbols, changes states according to predefined rules and decides whether the input is accepted or rejected.

Characteristics

- Reads input sequentially
- Changes states
- Follows transition rules
- Produces Accept/Reject decision

Basic Model



Key Idea

An automaton is a mathematical model, not a physical machine.

Historical Background of Automata Theory

- **1936 – Alan Turing**
 - Proposed Turing Machine
 - Mathematical Model of Computation
- **1943 – McCulloch & Pitts**
 - Mathematical Model of Artificial Neurons
- **1956 – Noam Chomsky**
 - Developed Chomsky Hierarchy
 - Classified Formal Languages and Grammars
- **1956–1960 – Stephen Kleene**
 - Introduced Regular Expressions
 - Contributed to Finite Automata Theory
- **1979 – Hopcroft & Ullman**
 - Published Influential Work on Automata and Formal Languages

Importance

These contributions became the foundation of:

- Compiler Design
- Programming Languages
- Artificial Intelligence
- Formal Verification

Why Should We Study Automata Theory?

Importance of Automata Theory

Automata Theory provides the mathematical foundation for designing computational systems that process, recognise, and validate structured inputs.

Major Areas of Application

- Compiler Design
- Programming Language Design
- Artificial Intelligence
- Natural Language Processing (NLP)
- Pattern Matching
- Cyber Security
- Digital Circuit Design
- Software Testing and Verification

Key Learning Outcome

Automata helps us understand how computers process information logically and systematically.

1. Compiler Design

- Token Recognition
- Lexical Analysis
- Syntax Analysis

2. Text Editors

- Syntax Highlighting
- Auto Completion

3. Search Engines

- Pattern Matching
- Query Processing

4. Artificial Intelligence

- Speech Recognition
- Natural Language Processing

5. Cyber Security

- Intrusion Detection
- Malware Detection

6. Networking

- Protocol Verification
- Packet Processing

Key Idea

Automata concepts are used wherever systems need pattern recognition and decision making.

How Does an Automaton Work?

Basic Working Principle

An automaton performs computation through a sequence of state transitions.

Processing Steps

- 1 Read an input symbol.
- 2 Move to another state according to transition rules.
- 3 Repeat until all symbols are processed.
- 4 Accept or Reject the input.

Working Model

Input $\rightarrow$ Current State $\rightarrow$ Transition Rule $\rightarrow$ Next State $\rightarrow$ Accept / Reject

Key Point

An automaton reads one input symbol at a time.

Concept of States

What is a State?

A state represents the current condition or status of computation.

During processing, an automaton changes from one state to another depending on the input symbol.

Types of States

- Initial State
- Intermediate States
- Final (Accepting) State

State Representation

$\rightarrow q_0$

$q_0 \rightarrow q_1 \rightarrow q_2$

- q_0 : Initial State
- q_2 : Final (Accepting) State

Key Idea

A state represents the current situation of an automaton during computation.

Automata Theory in Compiler Design

Compiler Phase	Automata Concept
Lexical Analysis	Regular Expressions, Finite Automata
Syntax Analysis	Context-Free Grammar
Parsing	Pushdown Automata
Semantic Analysis	Attribute Rules
Code Generation	Intermediate Representation

Relationship

Regular Expression



Finite Automata



Lexical Analyzer

Context-Free Grammar



Parser

Key Point

Automata Theory is the mathematical backbone of Compiler Design.

Real-Life Case Study: ATM Machine as an Automaton

ATM Transaction Process

An ATM performs a sequence of predefined operations based on user inputs. Each operation changes the system from one state to another.

Steps

- 1 Insert ATM Card
- 2 Verify Card
- 3 Enter PIN
- 4 Validate PIN
- 5 Select Transaction
- 6 Process Request
- 7 Dispense Cash
- 8 End Session

State Flow

Start → Insert Card → Verify Card → Enter PIN → Validate PIN → Transaction → Cash Dispensed → End

Key Idea

Each ATM operation represents a state transition in computation.

Examples of State-Based Systems

- Mobile Phone Lock
- Traffic Signal
- Lift (Elevator)
- Vending Machine
- Washing Machine
- Login Authentication
- Online Ticket Booking

Common Characteristics

All these systems:

- Receive Input
- Change States
- Follow Rules
- Produce Output

Key Idea

Many real-world systems behave like automata.

What is a Computational Model?

Definition

A computational model is a mathematical representation of a system that performs computation according to predefined rules.

Purpose

- Solve Computational Problems
- Process Information
- Recognise Patterns
- Accept or Reject Inputs

Examples

- Finite Automata (FA)
- Pushdown Automata (PDA)
- Turing Machine (TM)

Concept

Real System $\rightarrow$ Mathematical Model $\rightarrow$ Computation

What Does an Automaton Recognise?

Automata Recognise Languages

An automaton processes an input string and determines whether the string belongs to a particular language.

Language = Set of Valid Strings

Example:

$$L = \{ab, abb, abbb, \dots\}$$

Accepted

- ✓ ab
- ✓ abb
- ✓ abbb

Rejected

- × aa
- × ba
- × baba

Key Idea

An automaton checks whether a string follows the rules of a language, not its meaning.

Key Concepts Covered

- Why Automata Theory?
- Concept of Computation
- Automaton
- Historical Background
- Applications
- Computational Models
- Language Recognition
- Compiler Design Connection

Learning Outcomes

Students should now be able to:

- Explain the importance of Automata Theory.
- Define computation.
- Define an automaton.
- Identify real-life applications.
- Explain the connection between Automata Theory and Compiler Design.

Important Statement

Automata Theory is the mathematical foundation of Compiler Design.

How Does a Computer Process Input?

- A computer does not understand the meaning of the input.
- It processes every input according to predefined rules.

Example

Input String: abc

Processing Sequence

Read 'a' → Change State → Read 'b' → Change State → Read 'c' → Change State → End → Accept / Reject

Key Observation

- Reads one symbol at a time.
- Makes decisions based on predefined rules.
- Does not understand the meaning of the input.

Every Computational System Follows Rules

ATM

- Validates PIN
- Approves or Rejects Transaction

Compiler

- Checks Program Syntax

Login System

- Validates Username and Password

Email System

- Validates Email Format

General Model

Input → Rules → Decision

Key Idea

Computers make decisions by following predefined rules.

Basic Components of an Automaton

Fundamental Components

- **Input Alphabet (Σ)** – Set of input symbols.
- **States (Q)** – Different stages of computation.
- **Initial State (q_0)** – Starting state.
- **Final States (F)** – Accepting states.
- **Transition Function (δ)** – Defines movement between states.

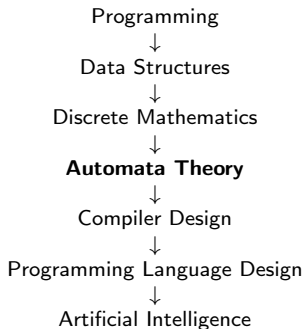
Conceptual Representation

Input $\rightarrow$ Current State $\rightarrow$ Transition $\rightarrow$ New State

Key Point

Every automaton is completely defined by these five components.

Learning Progression



Observation

Automata Theory connects mathematical concepts with practical software development.

Key Point

Strong Automata concepts make Compiler Design easier to understand.

In Today's Lecture, We Learned

- Need for Automata Theory
- Concept of Computation
- Automaton as an Abstract Machine
- Historical Development
- Applications of Automata
- Computational Models
- Rule-Based Processing
- Components of an Automaton
- Compiler Design Connection

Remember

Automata Theory provides the mathematical foundation for designing systems that recognise and process formal languages.

AUTOMATA THEORY

Automata Theory

Computation Automaton Applications

Input $\rightarrow$ Process $\rightarrow$ Output

Major Applications

- Compiler Design
- Artificial Intelligence
- Natural Language Processing (NLP)
- Search Engines
- Cyber Security

Key Idea

Automata Theory provides a mathematical model to describe computation and recognise formal languages.

Identify the State-Based System

Which of the following behave like an automaton?

- ATM Machine
- Traffic Signal
- Mobile Phone Lock
- Lift (Elevator)
- Washing Machine

Think–Pair–Share Activity

For any one system identify:

- Input
- States
- State Transition
- Output

Activity Model

System $\rightarrow$ Input $\rightarrow$ States $\rightarrow$ Output

Answer the Following Questions

- 1 What is Computation?
- 2 What is an Automaton?
- 3 Why is Automata Theory important?
- 4 Can a computer understand the C language directly?
- 5 Name any two real-life applications of Automata Theory.
- 6 Which compiler phase uses Finite Automata?

Purpose

Self-evaluation and concept checking.

After Completing This Lecture, Students Should Be Able To

- Explain the importance of Automata Theory.
- Define Computation.
- Define an Automaton.
- Describe the role of States in Computation.
- Identify real-life applications of Automata.
- Explain the relationship between Automata Theory and Compiler Design.

Achievement

Students now have the foundation required for learning DFA, NFA, Regular Expressions, Context-Free Grammar, and Pushdown Automata.

Conclusion & Preview of Next Lecture

Lecture Summary

Today we learned:

- Importance of Automata Theory
- Concept of Computation
- Automaton as an Abstract Machine
- Historical Development
- Applications of Automata
- Computational Models
- Compiler Design Connection

Next Lecture

Basic Terminology of Automata Theory

- Alphabet (Σ)
- Symbols
- Strings
- Empty String (ϵ)
- Length of String
- String Operations

Remember

Strong basics lead to a strong understanding of advanced Automata concepts.